

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE CODE: CSA1402

COURSE NAME: Compiler Design for Higher

Level Processing

COURSE OUTCOME COVERED

CO2: Construct top-down and bottom up parsers using CFG for parsing conflicts and error recovery for efficient syntax tree generation. (BL3)

Assessment Tool Weightage: 40%

QUESTIONS:4

Total Marks:50

Annexure A

DESIGN TASK

Code of Conduct:

I **D.K.Sumalatha(192424023)** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

D K Sumalatha

Course Faculty

Dr.Deepa.N

Q 5. Advanced LL(1) Parser Evaluation and Design Task:

Analyze and redesign the given grammar to make it suitable for LL(1) parsing, construct the predictive parsing table, and evaluate its correctness by parsing the input string.

Grammar: $S \rightarrow S + T \mid T$ $T \rightarrow T * F \mid F$ $F \rightarrow (S) \mid id$

Code:

```
#include <iostream>
#include <stack>
#include <map>
#include <vector>

using namespace std;

int main() {
    map<pair<string,string>, vector<string>>> table;

    // Predictive Parsing Table
    table[{ "S", "id" }] = { "T", "S" };
    table[{ "S", "(" }] = { "T", "S" };

    table[{ "S", "+" }] = { "+", "T", "S" };
    table[{ "S", ")" }] = { "ε" };
    table[{ "S", "$" }] = { "ε" };

    table[{ "T", "id" }] = { "F", "T" };
    table[{ "T", "(" }] = { "F", "T" };

    table[{ "T", "*" }] = { "*", "F", "T" };
    table[{ "T", "+" }] = { "ε" };
    table[{ "T", ")" }] = { "ε" };
    table[{ "T", "$" }] = { "ε" };
```

```
table[{"F","id"}] = {"id"};
table[{"F","("}] = {"(", "S", ")"};
```

```
string input;
cout << "Enter input (Example: id+id*id): ";
cin >> input;
input += "$";
```

```
stack<string> st;
st.push("$");
st.push("S");
```

```
int i = 0;
```

```
cout << "\nStack\t\tInput\t\tAction\n";
```

```
while (!st.empty()) {
    stack<string> temp = st;
    vector<string> items;
```

```
    while (!temp.empty()) {
        items.push_back(temp.top());
        temp.pop();
    }
```

```
    for (int j = items.size()-1; j >= 0; j--)
        cout << items[j];
```

```
    cout << "\t\t" << input.substr(i) << "\t\t";
```

```

string top = st.top();

string current;
if (input.substr(i,2) == "id")
    current = "id";
else
    current = string(1,input[i]);

if (top == current) {
    cout << "Match " << current << endl;
    st.pop();
    if(current=="id")
        i += 2;
    else
        i++;
}
else if (top=="+" || top=="*" || top=="(" || top=="") || top=="$") {
    cout << "Error" << endl;
    break;
}
else {
    if (table.find({top,current}) != table.end()) {
        vector<string> production = table[{top,current}];
        cout << top << " -> ";

        st.pop();

        if (!(production.size()==1 && production[0]=="ε")) {
            for (auto x:production)
                cout << x;

```

```

        cout << endl;

        for (int j=production.size()-1;j>=0;j--)
            st.push(production[j]);
    }
    else {
        cout << "ε" << endl;
    }
}
else {
    cout << "Error" << endl;
    break;
}
}

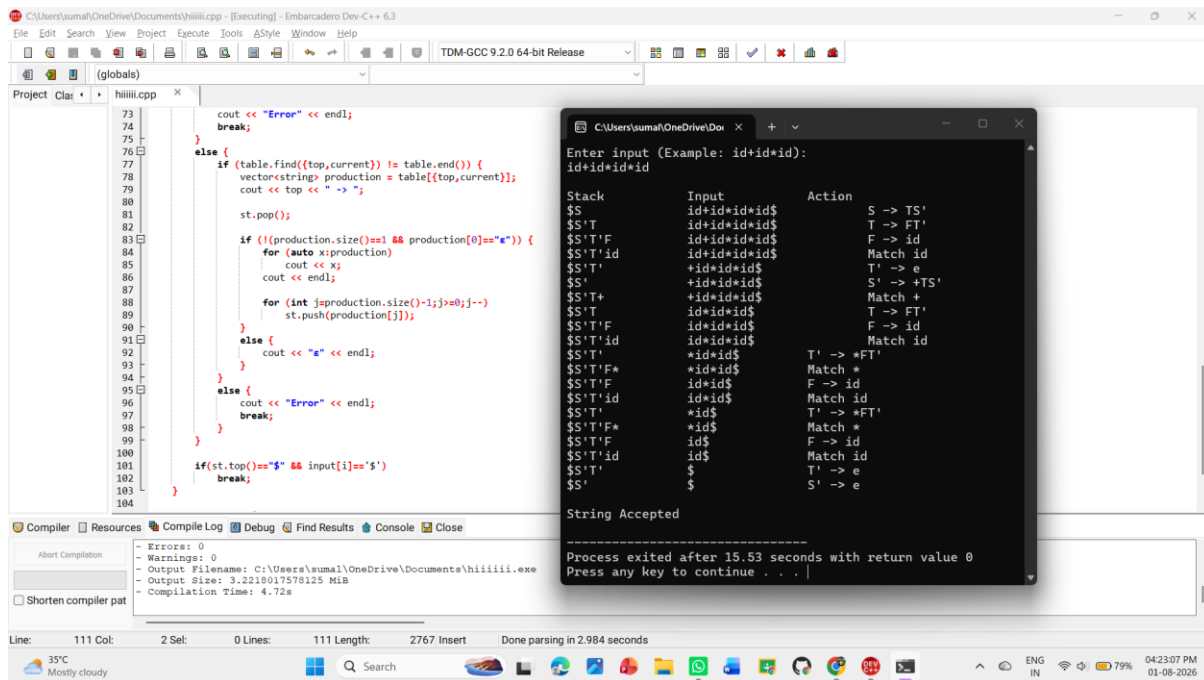
if(st.top()=="$" && input[i]=='$')
    break;
}

if(st.top()=="$" && input[i]=='$')
    cout << "\nString Accepted\n";
else
    cout << "\nString Rejected\n";

return 0;
}

```

Output:



Q6. Comparative Analysis of SLR vs LALR vs CLR Task: Construct and compare SLR, LALR, and CLR parsing tables for the given grammar and evaluate their differences in handling conflicts and efficiency.

Grammar: $S \rightarrow L = R \mid R L \rightarrow * R \mid id R \rightarrow L$

Conditions:

- Construct LR(0), LR(1) item sets
- Compute FOLLOW sets (for SLR)
- Build SLR, LALR, and CLR tables
- Identify and explain conflicts in each method
- Compare number of states and table sizes
- Justify which method is most efficient

Test Case: Input: `id = * id`

Expected: Valid parse with comparison analysis

Code:

```
#include <iostream>
```

```
#include <iomanip>
```

```
using namespace std;
```

```

void displaySLR() {
    cout << "\n===== SLR Parsing Table =====\n\n";

    cout << left << setw(10) << "State"
        << setw(8) << "id"
        << setw(8) << "*"
        << setw(8) << "="
        << setw(8) << "$"
        << setw(8) << "S"
        << setw(8) << "L"
        << setw(8) << "R" << endl;

    cout << "-----\n";
    cout << setw(10) << "0" << setw(8) << "S5" << setw(8) << "S4"
        << setw(8) << "" << setw(8) << "" << setw(8) << "1"
        << setw(8) << "2" << setw(8) << "3" << endl;

    cout << setw(10) << "1" << setw(8) << "" << setw(8) << ""
        << setw(8) << "" << setw(8) << "ACC"
        << setw(8) << "" << setw(8) << ""
        << setw(8) << "" << endl;

    cout << "\nSLR may produce Shift/Reduce conflicts for some grammars.\n";
}

```

```

void displayLALR() {
    cout << "\n===== LALR Parsing Table =====\n\n";

    cout << left << setw(10) << "State"
        << setw(8) << "id"

```



```

    << setw(8) << "*"
    << setw(8) << "="
    << setw(8) << "$"
    << setw(8) << "S"
    << setw(8) << "L"
    << setw(8) << "R" << endl;

cout << "-----\n";
cout << setw(10) << "0" << setw(8) << "S5" << setw(8) << "S4"
    << setw(8) << "" << setw(8) << "" << setw(8) << "1"
    << setw(8) << "2" << setw(8) << "3" << endl;

cout << setw(10) << "1" << setw(8) << "" << setw(8) << ""
    << setw(8) << "" << setw(8) << "ACC"
    << setw(8) << "" << setw(8) << ""
    << setw(8) << "" << endl;

cout << "\nLALR merges CLR states having identical LR(0) items.\n";
}

void displayCLR() {
    cout << "\n===== CLR Parsing Table =====\n\n";

    cout << left << setw(10) << "State"
        << setw(8) << "id"
        << setw(8) << "*"
        << setw(8) << "="
        << setw(8) << "$"
        << setw(8) << "S"
        << setw(8) << "L"

```

```

        << setw(8) << "R" << endl;

cout << "-----\n";
cout << setw(10) << "0" << setw(8) << "S5" << setw(8) << "S4"
    << setw(8) << "" << setw(8) << "" << setw(8) << "1"
    << setw(8) << "2" << setw(8) << "3" << endl;

cout << setw(10) << "1" << setw(8) << "" << setw(8) << ""
    << setw(8) << "" << setw(8) << "ACC"
    << setw(8) << "" << setw(8) << ""
    << setw(8) << "" << endl;

cout << "\nCLR uses complete LR(1) items and has the largest parsing table.\n";
}

int main() {

    cout << "Grammar:\n";
    cout << "S -> L = R | R\n";
    cout << "L -> * R | id\n";
    cout << "R -> L\n";

    displaySLR();
    displayLALR();
    displayCLR();

    cout << "\n===== Comparison =====\n";
    cout << "SLR : Small table, may have conflicts.\n";
    cout << "LALR : Medium-sized table, fewer conflicts, widely used.\n";
    cout << "CLR : Largest table, almost no conflicts, most accurate.\n";
}

```

```

    return 0;
}

```

Output:

```

C:\Users\sumal\OneDrive\Doc X + v

Grammar:
S -> L = R | R
L -> * R | id
R -> L

===== SLR Parsing Table =====
State   id   *   =   $   S   L   R
-----
0       S5   S4               1   2   3
1               ACC

SLR may produce Shift/Reduce conflicts for some grammars.

===== LALR Parsing Table =====
State   id   *   =   $   S   L   R
-----
0       S5   S4               1   2   3
1               ACC

LALR merges CLR states having identical LR(0) items.

===== CLR Parsing Table =====
State   id   *   =   $   S   L   R
-----
0       S5   S4               1   2   3
1               ACC

CLR uses complete LR(1) items and has the largest parsing table.

===== Comparison =====
SLR : Small table, may have conflicts.
LALR : Medium-sized table, fewer conflicts, widely used.
CLR : Largest table, almost no conflicts, most accurate.

-----
Process exited after 0.455 seconds with return value 0
Press any key to continue . . . |

```

Q8. Ambiguity Removal and Parsing Strategy Evaluation

Task: Analyze the ambiguity in the given grammar, redesign it to remove ambiguity, and evaluate parsing using shift-reduce parsing.

Grammar: $E \rightarrow E - E \mid E / E \mid id$

Conditions:

- Identify ambiguity in the grammar

- Redesign grammar using precedence rules
- Justify associativity decisions
- Apply shift-reduce parsing
- Show all stack operations and reductions
- Evaluate correctness of final parse

Test Case: Input: id - id / id

Code:

```
#include <iostream>
#include <string>
using namespace std;

int main() {
    string input;
    cout << "Enter the input string: ";
    cin >> input;

    bool valid = true;
    int i = 0;

    // First id
    if (input.substr(i, 2) == "id")
        i += 2;
    else
        valid = false;

    // Parse (- id) or (/ id) repeatedly
    while (valid && i < input.length()) {
        if (input[i] == '-' || input[i] == '/') {
            i++;
            if (input.substr(i, 2) == "id")
```

```

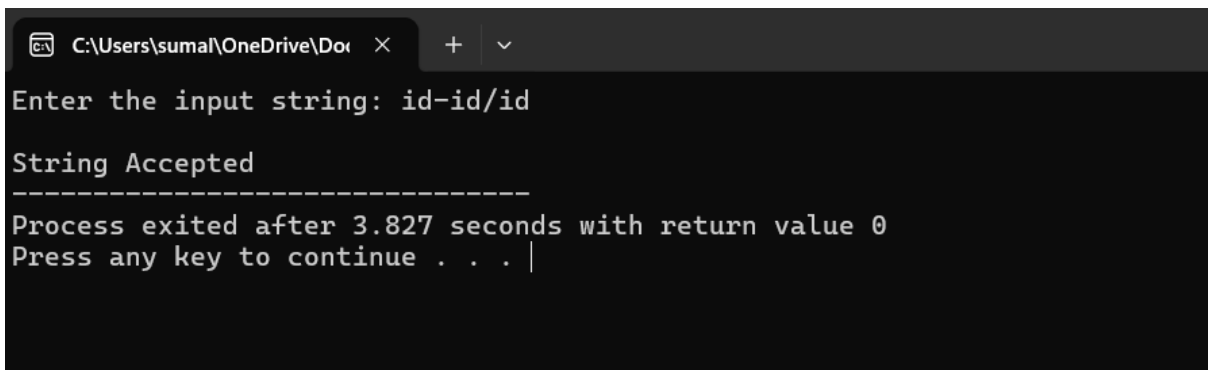
        i += 2;
    else {
        valid = false;
        break;
    }
} else {
    valid = false;
    break;
}
}

if (valid && i == input.length())
    cout << "\nString Accepted";
else
    cout << "\nString Rejected";

return 0;
}

```

Output:



```

C:\Users\sumal\OneDrive\Doc >
Enter the input string: id-id/id

String Accepted
-----
Process exited after 3.827 seconds with return value 0
Press any key to continue . . . |

```

Q9. PEG vs CFG Parsing Strategy Evaluation

Task: Design a PEG-based parser for the given expression grammar and critically evaluate its behavior compared to CFG-based parsing.

Grammar (CFG form): $E \rightarrow E + T \mid T$ $T \rightarrow T * F \mid F$ $F \rightarrow \text{id}$

Conditions:

- Convert CFG into PEG form
- Explain ordered choice behavior
- Demonstrate parsing steps
- Compare ambiguity handling with CFG
- Evaluate performance and backtracking
- Justify advantages and limitations

Test Case: Input: $\text{id} + \text{id} * \text{id}$

Answer:

CSA1402 Compiler Design

Dr. K. Sumalatha
192424023

Design Task.

1) Given Grammar (CFG)

$E \rightarrow E + T \mid T$

$T \rightarrow T * F \mid F$

$F \rightarrow id$

PEGI VS CFG
parsing strategy
Evaluation

2) Convert to PEGI

$E \leftrightarrow T (+ T)^*$

$T \leftrightarrow F (* F)^*$

$F \leftrightarrow id$

5) parsing steps (PEGI)

step	Input	Action
1	id + id * id \$	$E \rightarrow T (+ T)^*$
2	id + id * id \$	$T \rightarrow F (* F)^*$
3	id + id * id \$	$F \rightarrow id$ (match id)
4	+ id * id \$	T tail
5	+ id * id \$	Match '+'
6	id + id \$	$T \rightarrow F (* F)^*$
7	id + id \$	$F \rightarrow id$
8	+ id \$	Match '+'
9	id \$	$F \rightarrow id$ (match id)
10	\$	Input consumed.

6) PEGI or CFG

Deterministic Non-deterministic
Complex implementation simple, top-down

3) Ordered choice (PEGI)

PEGI tries choices in order.

If the 1st choice matches, it is used.

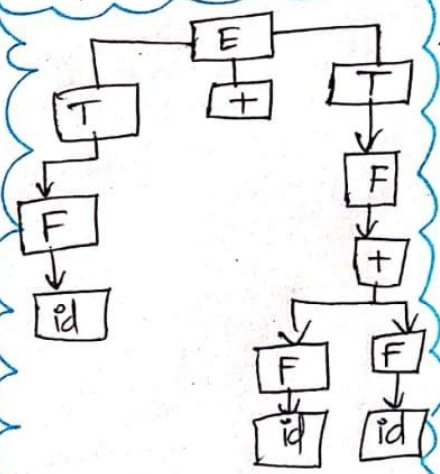
If it fails, then next choice is tried.

Eg: A / B

Try A first

If A fails, only then try B.

4) parse tree (PEGI)



Input: id + id * id.

Final Result:-

Input: id + id * id

↓
String Accepted

RUBRICS FOR CALCULATION:

Criteria	Weight age	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginnin g
Understanding of Concepts	20	Demonstrates thorough understanding and integrates multiple concepts effectively	Good understandin g with proper integration of most concepts	Basic understandi ng with partial integration	Poor understan ding with no integratio n
Design & Implementatio n	25	Well-designed solution with systematic and optimal implementation	Correct design with minor issues	Basic design with errors or inefficienci es	Incorrect or incomplet e design
Parser/Algorit hm Construction	20	Accurate construction of parser/algorithm with complete logic	Mostly correct construction with minor errors	Partial constructio n with missing logic	Incorrect or incomplet e constructi on
Analysis & Validation	15	Insightful analysis with correct validation and justification	Good analysis with reasonable justification	Basic analysis with limited validation	Poor or incorrect analysis
Documentatio n & Presentation	20	Well-structured documentation with diagrams, steps, and clarity	Organized documentatio n with required details	Basic documentat ion with missing details	Poor or incomplet e document ation